

ハンバーガーメニュー・ログアウトの実装

ログアウト機能とメニューのUI (App.vue の修正)

App.vue を修正して、画面の右上にメニューを表示し、そこからログアウトできるようにします。

```
<template>
  <div id="app">
    <header v-if="user" class="app-header">
      <div class="logo">勤怠システム</div>
      <div class="menu-container">
        <button @click="isMenuOpen = !isMenuOpen" class="hamburger">
          <span></span><span></span><span></span>
        </button>
        <div v-if="isMenuOpen" class="dropdown-menu">
          <p class="user-info">{{ user.userName }} さん</p>
          <hr>
          <button @click="logout" class="logout-btn">ログアウト</button>
        </div>
      </div>
    </header>
  </div>
</template>
```

```
<main>
  <LoginForm v-if="!user" @login-success="handleLoginSuccess" />
  <AttendanceBoard v-else :accountId="user.accountId" :userName="user.userName" />
</main>
</div>
</template>

<script setup>
import { ref } from 'vue';
import LoginForm from './components/LoginForm.vue';
import AttendanceBoard from './components/AttendanceBoard.vue';

const user = ref(null);
const isMenuOpen = ref(false);

const handleLoginSuccess = (userData) => {
  user.value = userData;
};

const logout = () => {
  if (confirm('ログアウトしますか?')) {
    user.value = null; // ユーザー情報を空にするだけで、v-ifによりログイン画面に戻ります
    isMenuOpen.value = false;
  }
};
</script>
```

```
<style scoped>
.app-header { display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 10px 20px;
  background: #333; color: white;
}
.hamburger { background: none; border: none; cursor: pointer; display: flex; flex-direction: column; gap: 5px; }
.hamburger span { display: block; width: 25px; height: 3px; background: white; }
.menu-container { position: relative; }
.dropdown-menu {
  position: absolute;
  right: 0; top: 40px;
  background: white;
  color: #333;
  border: 1px solid #ddd; border-radius: 5px;
  padding: 10px;
  min-width: 150px;
  z-index: 100;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}
.logout-btn { width: 100%;
  padding: 8px;
  background: #f44336; color: white;
  border: none; border-radius: 3px;
  cursor: pointer;
}
.user-info { font-size: 0.9rem; margin: 5px 0; }
</style>
```

打刻履歴の表示（バックエンド編：Javaの修正）

次に、過去の履歴を取得するAPIを作成します。

① AttendanceRecordRepository.java にメソッド追加

アカウントIDに基づいて、最新順に履歴を取得するメソッドを1行追加します。

```
public interface AttendanceRecordRepository extends JpaRepository<AttendanceRecord, Long> {  
    // accountIdで検索し、作成日時の降順（新しい順）で取得  
    List<AttendanceRecord> findByAccountIdOrderByCreatedAtDesc(String accountId);  
}
```

② AttendanceRecordService.java にメソッド追加

```
public List<AttendanceRecord> getHistory(String accountId) {  
    return repository.findByAccountIdOrderByCreatedAtDesc(accountId);  
}
```

③ AttendanceController.java の該当箇所を次のように修正

```
/**
 * 打刻履歴一覧を取得する
 */
@GetMapping("/history")
public ResponseEntity<List<AttendanceRecord>> getHistory(@RequestParam String accountId) {
    // Service側のメソッド名も、Serviceの実装に合わせて修正します
    List<AttendanceRecord> history = attendanceService.getHistory(accountId);
    return ResponseEntity.ok(history);
}
```

④ AttendanceRecord.java に定義とメソッドを追加

このクラス内でcreatedAtを定義していないと、いくら①～③を誤りなく修正・追加したところで、「定義の不整合」が起きて意味をなさなくなるので、ご注意ください。

```
// ... 他のインポート
import java.time.LocalDateTime;
import jakarta.persistence.PrePersist; // 追加：保存前に実行するための設定

@Entity
public class AttendanceRecord {
    // ... 既存の id, accountId, type などのフィールド

    // 追加：打刻日時を保持するフィールド
    private LocalDateTime createdAt;

    // 追加：データがデータベースに保存される直前に、現在時刻を自動セットする
    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
    }

    // Getter と Setter も忘れずに追加してください（Lombokを使っている場合は不要です）
}
```

打刻履歴の表示（フロントエンド編：Vueの修正）

AttendanceBoard.vue の下部に、取得した履歴をリスト表示する領域を追加します。

```
const history = ref([]);
const fetchHistory = async () => {
  try {
    const response = await apiClient.get(`/attendance/history?accountId=${props.accountId}`);
    history.value = response.data;
  } catch (error) {
    console.error('履歴取得失敗', error);
  }
};

// 打刻成功時にも履歴を更新するようにする
const punch = async (type) => {
  try {
    await apiClient.post(`/attendance/${type}?accountId=${props.accountId}`);
    fetchStatus();
    fetchHistory(); // ← これを追加
  } catch (error) { ... }
};
```



```
onMounted(() => {  
  fetchStatus();  
  fetchHistory(); // 画面表示時にも読み込む  
});
```

AttendanceBoard.vue の修正 (template部分)

```
<div class="history-section">  
  <h4>最近の履歴</h4>  
  <ul>  
    <li v-for="record in history" :key="record.id">  
      {{ record.type === 'CLOCK_IN' ? '入' : '出' }} :  
      {{ new Date(record.createdAt).toLocaleString('ja-JP') }}  
    </li>  
  </ul>  
</div>
```

この後の流れ

1. まず `App.vue` を書き換えて、ログアウトができるか確認してください。
2. 次に Java 側の `Repository` , `Service` , `Controller` , `Entity` を修正します。
3. 最後に `AttendanceBoard.vue` に履歴表示ロジックを追加します。

これで、単なる「打刻ボタン」から、その履歴の確認ができる「管理ツール」へと進化します。